

CURT Task Tracking & Development

Internal Chatbot: Memory Architecture R&D Specification

Author: Khaled (Memory & R&D Engineer)

Project: CURT Generative AI Team - Internal Chatbot

Date: May 2026

Data Scope: BAMOCAR Inverter Manuals, Emrax Motor (188-348) Manuals

1. Executive Summary

This document outlines the architectural decisions for the long-term memory system of the CURT Internal Chatbot (Task 2). The objective of this system is to dynamically track, extract, and summarize team problems, parameter adjustments, and major milestones derived from workshop troubleshooting sessions. The architecture is designed to integrate seamlessly with the LangChain RAG pipeline being developed by the Advanced RAG Engineer.

2. Architectural Decisions

Decision 1: The Trigger Mechanism

- **Selected Method:** Explicit Trigger (Human-in-the-Loop)
- **Justification:** Troubleshooting electromechanical systems (e.g., high-voltage inverter parameters, motor cooling thresholds) requires extreme precision. An explicit trigger (e.g., the user explicitly commanding the bot to "save this solution") guarantees that only verified, human-approved engineering solutions enter the team's permanent knowledge base. This prevents the LLM from autonomously logging experimental or unsafe parameter tweaks.
- **Implementation:** The memory-saving function will be built as a modular LangChain Tool, decoupling the "save" action from the conversation logic.

Decision 2: Memory Formatting

- **Selected Method:** Atomic Fact Extraction (Semantic Summarization)
- **Justification:** Rather than saving raw, messy conversation logs or enforcing brittle JSON schemas, the system will evaluate the resolved conversation thread and extract dense, self-contained semantic facts (e.g., "*Emrax 228 max RPM was limited to 4000 on May 12th to prevent overheating*"). These facts are highly flexible, avoid "missing field" errors, and are natively converted into embeddings, making them instantly compatible with the

existing RAG pipeline.

Decision 3: Storage Infrastructure

- **Selected Method:** Isolated Dual-Collection Database
- **Justification:** Team memories will be stored in a dedicated vector collection, strictly separated from the official BAMOCAR and Emrax manual embeddings. This isolation is critical. It ensures the LLM can differentiate between the absolute "Ground Truth" (manufacturer specifications) and the "Workshop Reality" (temporary fixes or team-specific tuning), preventing dangerous hallucinations during future troubleshooting.

3. Future R&D & Trials Scope

To continuously improve the system and reduce friction for the workshop engineers, the following architectures will be developed and tested in the background:

Trial A: Autonomous Agent Trigger (LangGraph)

- **Concept:** Transitioning from the Explicit Trigger to an autonomous evaluation loop.
- **Testing Plan:** Because the memory trigger is modular, we will deploy a background evaluator agent in "Shadow Mode". It will monitor chat sessions and print to the terminal when it detects a "resolved problem" state, without actually writing to the database. This allows us to verify the agent's accuracy before moving to a fully autonomous memory-saving system.

Trial B: Temporal Knowledge Graphs (Nodes & Edges)

- **Concept:** Upgrading from flat Atomic Facts to a relational Graph Database (e.g., Neo4j).
- **Testing Plan:** We will research mapping hardware states using nodes and edges (e.g., [BAMOCAR Inverter] --(Parameter: 380V)--> [Resolved: Fault Code 22]). This trial aims to build a deterministic, timestamped audit trail of the car's hardware adjustments. If successful, this will provide superior state-tracking over standard vector databases for highly complex mechatronic troubleshooting.